# Module: Variables, Datatypes, Operators, and I/O

**Phase:** Core Python
**Previous module:** Python Setup and First Programs
**Next module:** Flow Control

**Module Outcome:**

- Name a labelled box ( `student_name = "Ravi"` ) and know what type is inside
- Do money and marks maths with operators — and print a clear result
- Read from `input()` and write a line the user can actually understand

## What is a variable?

A **variable** is a name that points to a value stored in memory. Think of it as a labelled box: the label is `student_name`, the contents are `"Ravi Kumar"`.




**Where used?**
- **HDFC Bank** / **PhonePe** — balance, transfer amount, PIN flags
- College apps — roll, marks, fees as `int` / `float`
- **Employee Management Portal** — basic salary, HRA, CTC in lakhs
- **Swiggy** — cart line price × qty, GST totals
- **Apollo Hospitals** — patient age, ticket amount, `is_active` flags

```
student_name = "Ravi Kumar"   # str
roll_number = 101             # int
percentage = 87.5             # float
is_pass = True                # bool
```

Python uses **dynamic typing**: you do not declare `int x = 5` like Java. The type is decided by the value you assign, and can change (though good code avoids confusing reassignments).

```
amount = 100      # int
amount = 99.50    # now float — allowed but be clear in real projects
```

## Core datatypes

| Type | Example | Use in training projects |
|---|---|---|
| **int** | `42`, `-7`, `0` | Roll numbers, counts, quantities |
| **float** | `3.14`, `99.99` | Salary, expense amounts, percentages |
| **str** | `"hello"`, `'EMP001'` | Names, emails, IDs as text |

| Type | Example | Use in training projects |
|---|---|---|
| **bool** | `True`, `False` | Flags: logged in, active employee |
| **None** | `None` | Missing value: no manager assigned yet |

Check type:

```
print(type(roll_number))   # <class 'int'>
print(type(is_pass))       # <class 'bool'>
```

## Type conversion (casting)

User input always arrives as **string**. You must convert for math.

```
age_text = input("Enter age: ")   # e.g. user types 22
age = int(age_text)               # convert to int
next_year = age + 1
```

Invalid conversion crashes:

```
int("abc")   # ValueError
```

Always validate or use try/except (later module) in production apps.

## Operators — quick map

```
Arithmetic:  +  -  *  /  //  %  **
Comparison:  ==  !=  <  >  <=  >=
Logical:     and  or  not
Assignment:  =  +=  -=  *=  /=
Membership:  in  not in
Identity:    is  is not
```

## Input and output

**Output:**

```
name = "Ananya"
print("Hello", name)              # space between
print(f"Hello, {name}!")          # f-string — preferred
```

**Input:**

```
email = input("Enter work email: ")
print(f"You entered: {email}")
```

Everything from `input()` is `str` until you convert.

---

## Important Concepts and Definitions

| Term | Definition |
|---|---|
| **Variable** | Identifier bound to an object in memory |
| **Literal** | Fixed value written in code: `10`, `"hi"`, `True` |
| **Dynamic typing** | Type tied to object, not variable declaration |
| **Type conversion / casting** | Changing representation: `int("5")` → `5` |
| **Operator** | Symbol performing operation on operands |
| **Operand** | Value an operator acts on |
| **Expression** | Combines values/operators to produce result |
| **f-string** | Formatted string: `f"Total: {total}"` |
| **Modulo (`%`)** | Remainder after division |
| **Floor division (`//`)** | Integer division discarding fraction |
| **Truthy / Falsy** | Values that behave as True/False in conditions |

**Falsy values in Python:** `0`, `0.0`, `""`, `None`, `False`, empty collections (later)

## Step-by-Step Notes (With Annotated Code)

### Step 1: Variables for student record

```
# student_record.py — partial record (no input yet)

roll = 2026001
name = "Priya Nair"
marks_python = 78
marks_sql = 82
average = (marks_python + marks_sql) / 2   # float result

print("Roll:", roll)
print("Name:", name)
print(f"Average marks: {average:.2f}")     # 2 decimal places
```

**Note:**
- Wrong: Typo in a name (`salary = 100` then `print(salery)`) → `NameError`.
- Prefer: Spell names consistently; copy-paste the identifier when unsure.

Store Ravi's roll number, name, and two subject marks, then print his average to two decimal places.

```
roll = "2026002"
name = "Ravi"
python_marks = 81
sql_marks = 74
average = (python_marks + sql_marks) / 2
print(f"{name} ({roll}) average: {average:.2f}")
```

**Expected Output:**

```
Ravi (2026002) average: 77.50
```

Store food and travel expenses, compute the daily total, and print it formatted with ₹ to two decimal places.

```
food = 185.50
travel = 60.0
total = food + travel
print(f"Today's total: ₹{total:.2f}")
```

**Expected Output:**

```
Today's total: ₹245.50
```

## Step 2: Datatypes and None

```
employee_id = "E1024"        # str — IDs often kept as text (leading zeros)
years_experience = 3         # int
ctc_lakh = 4.5               # float — CTC in lakhs
is_remote = True             # bool
manager_id = None            # no manager for top-level employee

print(type(employee_id))     # <class 'str'>
print(manager_id is None)    # True — preferred None check
```

**Note:**
- Wrong: Checking None with `if x == None`.
- Prefer: `if x is None`.

Define Employee Management Portal employee fields for Priya Shah and print the type of each value.

```
emp_id = "E2048"
years = 2
ctc_lakhs = 6.5
is_remote = True
manager_id = None
print(type(emp_id), type(years), type(ctc_lakhs), type(is_remote), type(manager_id))
```

Set manager_id to None and is_remote to True, then print the None and bool identity checks.

```
manager_id = None
is_remote = True
print(manager_id is None)
print(is_remote is True)
```

**Expected Output:**

```
True
True
```

**Remember:** Check missing values with `x is None` (identity), not `x == None`.

### Step 3: Arithmetic operators

```
a, b = 17, 5

print(a + b)    # 22   addition
print(a - b)    # 12   subtraction
print(a * b)    # 85   multiplication
print(a / b)    # 3.4  true division (always float in Py3)
print(a // b)   # 3    floor division
print(a % b)    # 2    remainder — useful: even/odd → n % 2
print(a ** b)   # 1419857  power
```

**Expense tracker example:**

```
food = 450.50
travel = 120.00
total = food + travel
print(f"Today's spend: ₹{total:.2f}")
```

**Note:**
- Wrong: Expecting `5 / 2` to be `2` (it is `2.5` in Python 3); showing raw `0.1 + 0.2` floats in money UIs.
- Prefer: Use `//` for integer division; round for display (`f"{x:.2f}"`).

Apply all seven arithmetic operators to 25 and 4 and print each result.

```
a, b = 25, 4
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```

**Expected Output:**

```
29
21
100
6.25
6
1
390625
```

Split a ₹999 bill among 3 friends using division, floor division, and remainder.

```
bill = 999
friends = 3
print(bill / friends)   # exact share
print(bill // friends)  # whole rupees each
print(bill % friends)   # leftover as remainder
```

**Expected Output:**

```
333.0
333
0
```

### Step 4: Comparison and logical operators

```
marks = 72
passing = 35

print(marks >= passing)          # True
print(marks == 100)              # False

# Employee login check (simplified)
username_ok = True
password_ok = False
can_login = username_ok and password_ok
print(can_login)                 # False

# At least one condition
has_coupon = False
is_member = True
discount = has_coupon or is_member  # True
```

**Truth table (memorize for interviews):**

| A | B | A and B | A or B |
|---|---|---|---|
| T | T | T | T |
| T | F | F | T |
| F | T | F | T |
| F | F | F | F |

**Note:**
- Wrong: Using `=` inside a condition (`if x = 5`) — assignment, not comparison.
- Prefer: `==` for comparison.

Given marks 68, print pass, distinction, and fail flags using comparison operators.

```
marks = 68
pass_mark = 35
print("pass:", marks >= pass_mark)
```

```
print("distinction:", marks >= 75)
print("fail:", marks < pass_mark)
```

Use and/or to evaluate login eligibility and discount eligibility, then print both results.

```
user_ok = True
pin_ok = False
can_login = user_ok and pin_ok
print("can_login:", can_login)

is_member = False
has_coupon = True
print("discount eligible:", is_member or has_coupon)
```

## Step 5: Assignment operators

```
balance = 1000
balance += 250    # same as balance = balance + 250  → 1250
balance -= 400    # 850
balance *= 2      # 1700

task_count = 0
task_count += 1   # common pattern when adding tasks
```

Start with balance ₹8000, deposit ₹1500, withdraw ₹2000, and print the final balance.

```
balance = 8000
balance += 1500
balance -= 2000
print(f"Final balance: ₹{balance}")
```

**Expected Output:**

```
Final balance: ₹7500
```

Build a month total by adding three expenses with += and print the result.

```
month_total = 0
month_total += 120
month_total += 85.5
month_total += 40
print(f"Month total: ₹{month_total}")
```

**Common Error:** Writing `input("Amount: ") + 100` without casting — `input()` is always `str`, so you get `TypeError` (or string concat). Convert with `float(...)` / `int(...)` first.

## Step 6: Membership and identity

```
email = "ravi@company.com"
print("@" in email)          # True
print("gmail" in email)      # False
```

```
status = None
if status is None:
    print("Status not set yet")
```

Use `==` for value comparison; `is` mainly for `None` and small integers cache (interview nuance — avoid `is` for strings/content).

Check whether priya@ksr.in contains @ and .in using membership operators.

```
email = "priya@ksr.in"
print("@" in email)
print(".in" in email)
```

If manager is None, print that no manager is assigned yet.

```
manager = None
if manager is None:
    print("No manager assigned yet")
```

## Step 7: input() and type conversion

```
# expense_entry.py

print("--- Expense Tracker ---")
category = input("Category (food/travel/other): ")
amount_str = input("Amount in ₹: ")

amount = float(amount_str)   # convert for calculations
print(f"Saved: {category} → ₹{amount:.2f}")
```

### Safe-ish integer input pattern (beginner):

```
roll_str = input("Roll number: ")
if roll_str.isdigit():
    roll = int(roll_str)
    print("Roll as int:", roll)
else:
    print("Invalid roll – digits only")
```

**Note:**
- Wrong: Treating `input()` as a number (`input("N: ") + 1`) or casting `int("12.5")` directly.
- Prefer: `int(input(...))` / `float(input(...))`; for `"12.5"` use `float` first (then `int` if needed).

Simulate saving a food expense of ₹250 and print the formatted saved message.

```
category = "food"
amount = float("250")
print(f"Saved: {category} → ₹{amount:.2f}")
```

Validate roll text with isdigit, convert to int if valid, or print Invalid roll.

```
roll_text = "2026001"
if roll_text.isdigit():
```

```
    roll = int(roll_text)
    print(roll)
else:
    print("Invalid roll")
```

### Step 8: f-strings in depth

```
name = "Karthik"
role = "Backend Intern"
salary = 25000

# Basic
print(f"{name} works as {role}")

# Expressions inside braces
print(f"Annual (approx): ₹{salary * 12:,}")   # ₹300,000 formatting

# Alignment (reports)
print(f"{'Item':<15}{'Amount':>10}")
print(f"{'Food':<15}{450.5:>10.2f}")
```

**Note:**
- Wrong: Concatenating str + int (`"Age: " + 22`) → `TypeError`.
- Prefer: `f"Age: {22}"` or `str(22)`.

Print an Employee Management Portal offer line with name, role, and comma-formatted monthly salary.

```
name = "Ravi"
role = "Intern"
salary = 25000
print(f"{name} — {role} — ₹{salary:,}/month")
```

Print a Swiggy canteen receipt with left-aligned items and right-aligned prices.

```
print(f"{'Item':<10}{'Price':>8}")
print(f"{'Chai':<10}{'₹20':>8}")
print(f"{'Samosa':<10}{'₹25':>8}")
```

### Step 9: Mini CLI calculator

```
# simple_calculator.py

print("=== Student Marks Calculator ===")
m1 = float(input("Python marks: "))
m2 = float(input("SQL marks: "))
m3 = float(input("HTML marks: "))

total = m1 + m2 + m3
average = total / 3

print(f"Total: {total:.2f}")
print(f"Average: {average:.2f}")
print(f"Pass?" , average >= 35)   # preview of flow control
```

Compute total and average for three subject marks and print pass status.

```
m1, m2, m3 = 78, 82, 91
total = m1 + m2 + m3
average = total / 3
print(f"Total: {total}")
print(f"Average: {average:.2f}")
print("Pass:", average >= 35)
```

Calculate Swiggy line total for price × qty with 18% GST and print the result.

```
price = 499.0
qty = 2
line_total = price * qty * 1.18
print(f"Line total with GST: ₹{line_total:.2f}")
```

**Expected Output:**

```
Line total with GST: ₹1177.64
```

---

## Practice Exercises

### Basic

1. Create variables for a student: name, age, course, fees (float). Print with f-string.

**Model answer:**

```
name = "Ananya"
age = 21
course = "Full Stack Python"
fees = 45000.0
print(f"{name}, age {age}, enrolled in {course}. Fees: ₹{fees}")
```

2. Convert `"100"` and `"3.14"` to int and float; print types.

**Model answer:**

```
n = int("100")
x = float("3.14")
print(n, type(n))
print(x, type(x))
```

3. Evaluate: `10 + 5 * 2`, `20 // 3`, `20 % 3`, `2 ** 4`.

**Model answer:**

```
print(10 + 5 * 2)  # 20 — multiply first
print(20 // 3)     # 6 — floor division
print(20 % 3)      # 2 — remainder
print(2 ** 4)      # 16 — power
```

4. Ask user for name and city; print greeting.

**Model answer:**

```
name = input("Name: ")
city = input("City: ")
print(f"Namaste, {name} from {city}!")
```

## Intermediate

1. **Employee portal:** Input basic salary and HRA (float); print total compensation.

**Model answer:**

```
basic = float(input("Basic salary: "))
hra = float(input("HRA: "))
total = basic + hra
print(f"Total compensation: ₹{total:.2f}")
```

2. Check if email contains `@` and `.com` using `in` and `and`.

**Model answer:** (boolean expression only — no `if` yet)

`python email = input("Email: ") looks_ok = "@" in email and ".com" in email print(looks_ok) print(f"Contains @ and .com: {looks_ok}")`

3. Use `+=` to simulate adding three expenses to a running total starting at 0.

**Model answer:**

```
total = 0
total += 120   # chai + snack
total += 450   # lunch
total += 80    # metro
print(f"Running total: ₹{total}")
```

4. Format a receipt table (3 items, amounts, grand total) with aligned f-strings.

**Model answer:**

```
item1, amt1 = "Samosa", 40.0
item2, amt2 = "Thali", 180.0
item3, amt3 = "Lassi", 60.0
grand = amt1 + amt2 + amt3
print(f"{'Item':<12}{'Amount':>8}")
print(f"{item1:<12}{amt1:>8.2f}")
print(f"{item2:<12}{amt2:>8.2f}")
print(f"{item3:<12}{amt3:>8.2f}")
print(f"{'TOTAL':<12}{grand:>8.2f}")
```

## Interview-Oriented

1. Difference between `/` and `//` in Python 3 with examples.

**Model answer:** `/` is true division and returns a **float** (`7 / 2` → `3.5`). `//` is floor division toward −∞ for ints (`7 // 2` → `3`). Use `/` for real averages; `//` for whole groups (e.g. how many full dozens).

2. What is dynamic typing? Is Python strongly typed? (Yes — no implicit str+int.)

**Model answer: Dynamic typing:** you do not declare types; a name can hold an `int` then later a `str`. **Strongly typed:** incompatible operations fail — `"5" + 5` raises `TypeError`; you must cast explicitly (`int("5") + 5`). So: dynamic *and* strong.

3. Explain `is` vs `==` with `None` example.

**Model answer:** `==` compares **values**. `is` compares **object identity** (same object in memory). For `None`, the idiom is `x is None` / `x is not None`, not `x == None`, because there is only one `None` object and identity is the clear check.

4. Why does `0.1 + 0.2 == 0.3` evaluate False? (Float representation — mention rounding.)

**Model answer:** Binary floating point cannot represent 0.1 and 0.2 exactly, so their sum is a tiny bit off from 0.3. Compare with rounding or a tolerance: `round(0.1 + 0.2, 10) == 0.3`, or use `decimal.Decimal` for money later. Never assume float equality for currency without care.

---

## Mini-Project (Assignment): Swiggy CLI Canteen Calculator

**Title:** One-session calculator with GST + validation
**Lab kit (build here):** 15_Lab_Assets/06_cli_calculator/
**Also use:** 07_casting_bugs
**Later (files module):** 01_python_expense_tracker — CSV export after you learn file handling

### What to build

1. Read item **price** and **qty** from the user (floats / ints via casting)
2. Compute `subtotal = price * qty`, then **GST 5%**, then grand total
3. If price or qty `<= 0`, print an error and do not compute
4. Print a ₹ receipt line with 2 decimal places
5. Loop for **3** items in one run; keep `day_total` with `+=`

**Sample run:**

```
--- Canteen Calculator ---
Price: 40
Qty: 2
✓ Item: ₹80.00 + GST ₹4.00 = ₹84.00
...
Day total: ₹210.00
```

### Stretch

- Reject non-numeric input without crashing (reuse casting-bugs ideas).
- Print a mini table of all three items with grand total.

### Done when

- [ ] Lab kit canteen calculator ticket passes
- [ ] Three valid items update `day_total` correctly
- [ ] Amount/`qty` `<= 0` prints an error and is not added
- [ ] Output uses `₹` and two decimal places

## Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. Should employee ID be stored as `int` or `str`? When does leading zero matter?
2. Why do banking apps show ₹ symbols in UI but store numbers as float/decimal in backend?
3. What happens if a user presses Enter without typing anything in `input()`?
4. Is `not in` the opposite of `in`? Give example with email domain check.
5. where have you used variables in Excel formulas — same idea?

**Q1. Name Python's basic data types.**
**A:** int, float, str, bool, NoneType; plus list, dict, tuple, set at collection level.

**Q2. What is an f-string?**
**A:** Literal prefixed with `f` allowing embedded expressions `{expr}` evaluated and formatted in string.

**Q3. Difference between `and` and `or`?**
**A:** `and` true only if both operands true; `or` true if at least one true.

**Q4. What does `%` operator do?**
**A:** Modulo — remainder of division; used for even/odd, cycles, hash buckets.

**Q5. Is Python strongly typed?**
**A:** Yes — does not silently convert incompatible types (no automatic str+int); dynamic means types not declared upfront.

---